jazz-guru

Architecture

Local Claude agent harness for music + text (MusicXML · MIDI · WAV)

Python 3.12 · FastAPI · Anthropic Claude · Postgres + pgvector · Redis / RQ

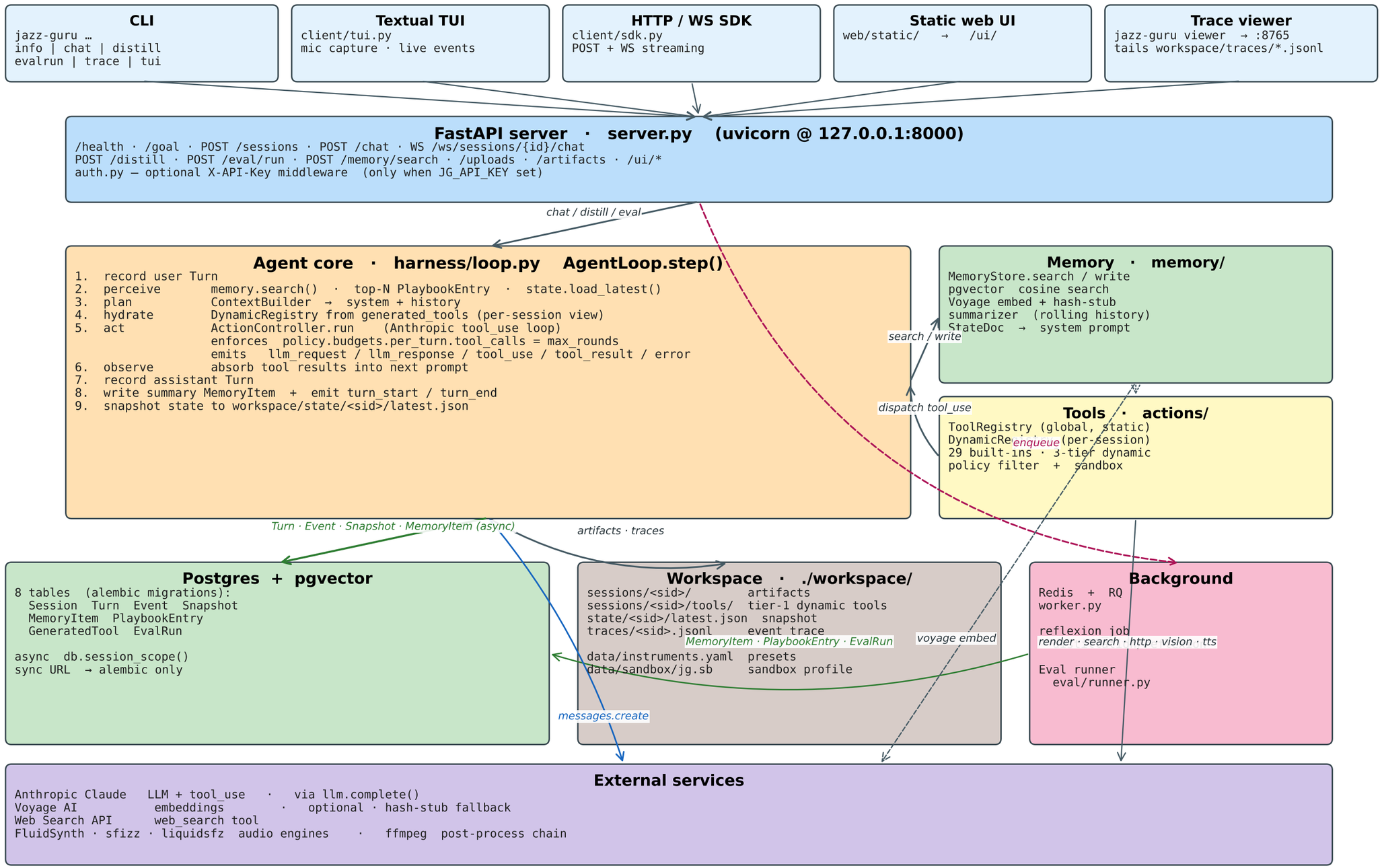
Contents

15 pages · rendered from src/jazz_guru/ + CLAUDE.md

`open with any PDF viewer  ·  re-render via `.venv/bin/python workspace/scratch/architecture_diagram.py``

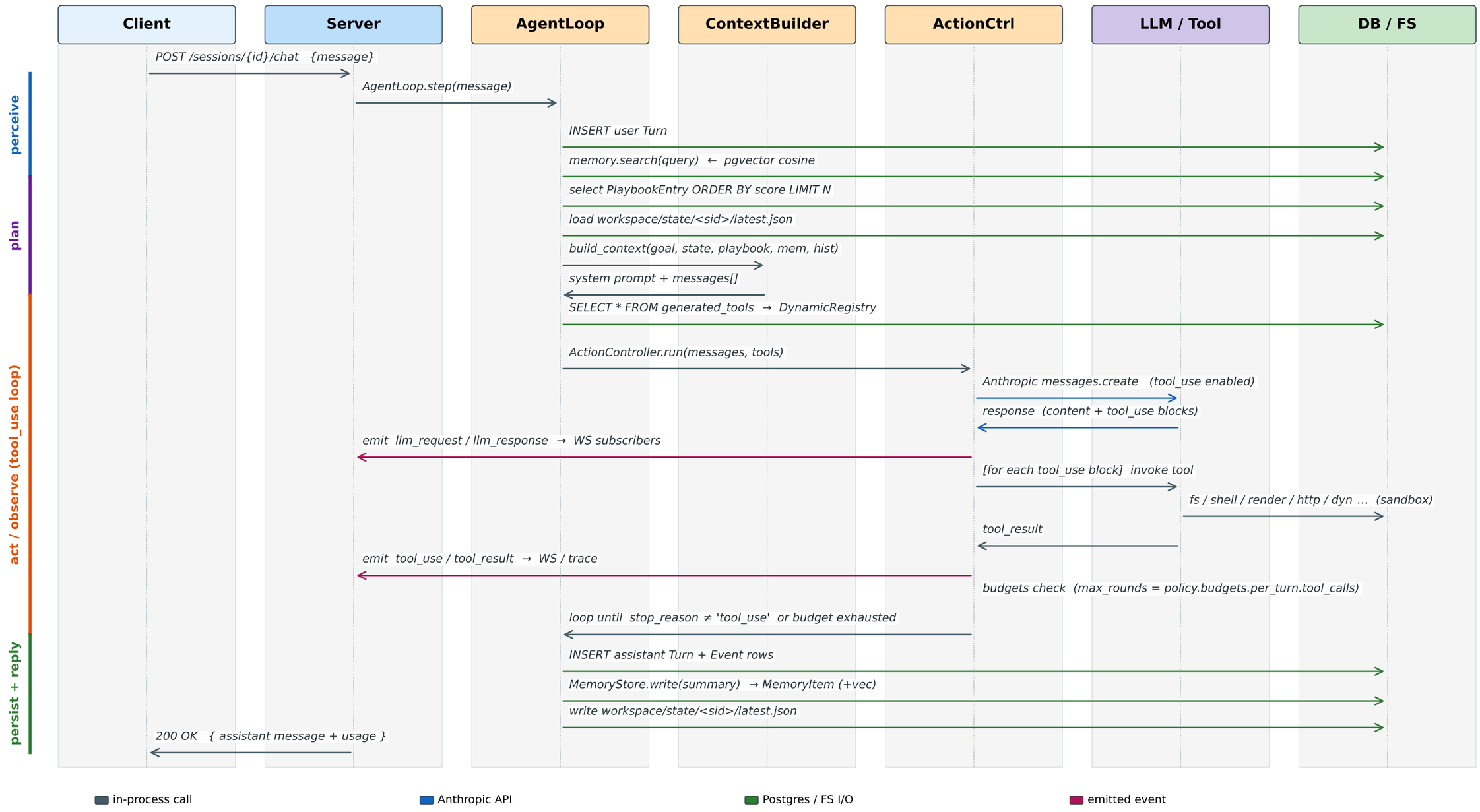
System overview

Clients invoke the FastAPI server, which drives the agent loop. Each turn touches Postgres, the workspace filesystem, and external services.



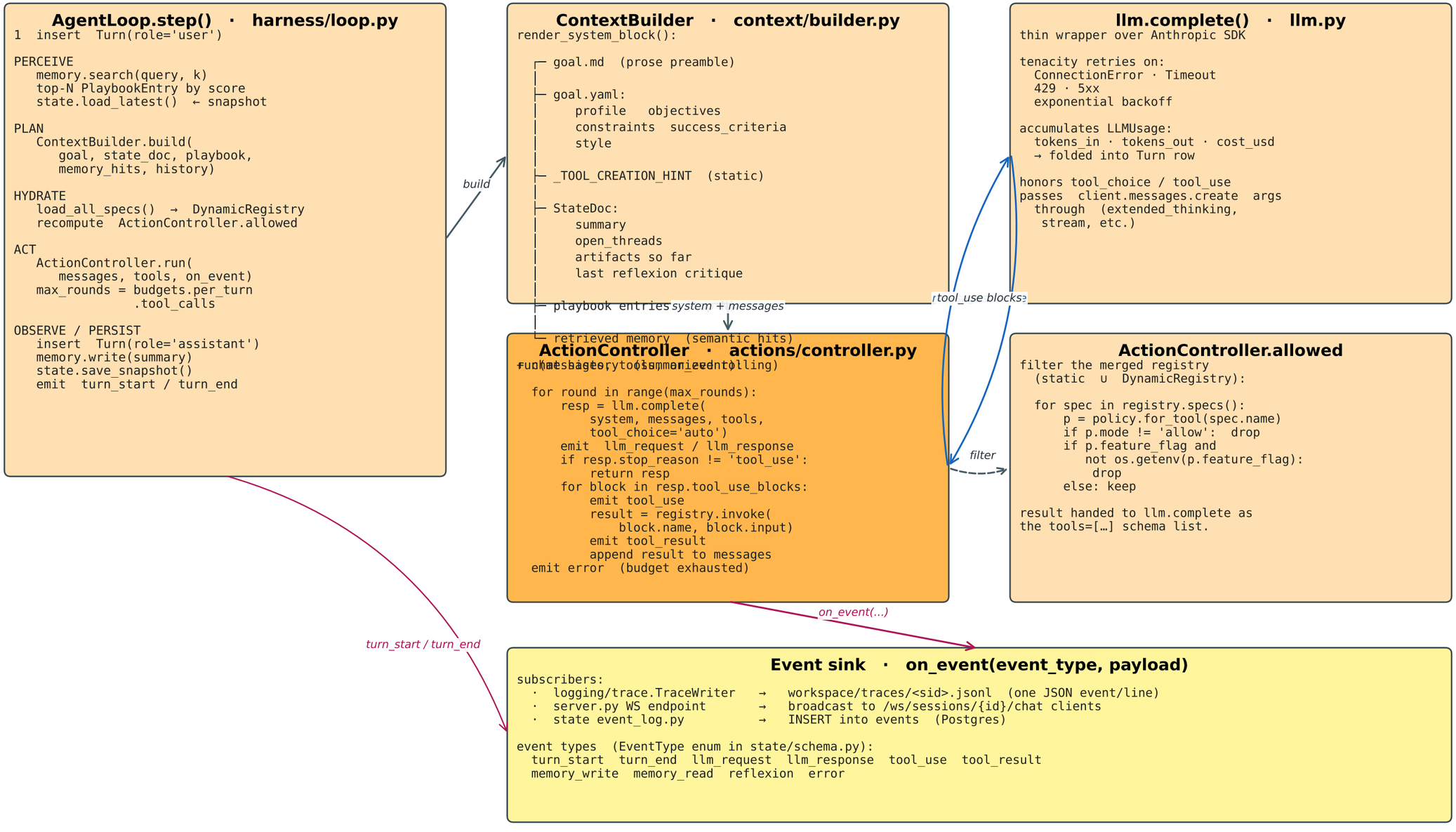
Turn lifecycle

One AgentLoop.step() call — what happens when the user sends a chat message.



Agent core internals

Loop · context · controller · llm — the seam every chat() call passes through.



Tool system + built-in catalogue

Global ToolRegistry merged each turn with a per-session DynamicRegistry; filtered by policy; dispatched via ActionController.

ToolRegistry (process-global, static) · actions/registry.py

```
@registry.register(name='fs_read',
                   description=...,
                   input_model=FsReadInput)
def fs_read(ctx, **kwargs):  ...

register_all() imports every tools/*.py module
  → decorators fire → registry is populated
  → idempotent on re-call
```

merge →

DynamicRegistry (per-session) · actions/dynamic.py

```
view = merge( static_registry, generated_tools )
  generated_tools where scope='global'  AND  rows tier-1-loaded
  for current session via store.load_all_specs()

specs carry: name · description · JSON-schema · source · sha256
default execution = `python -I` subprocess  (cwd = session ws)
  stdin: JSON kwargs   stdout: JSON result   timeout: python_exec policy
  inproc fallback for trusted helpers
```

Built-in tools · src/jazz_guru/actions/tools/ (29 names registered)

Filesystem

fs.py	fs_read	fs_write	fs_list

Shell / Python

shell.py	shell	(subprocess; sandboxed)
python_exec.py	python_exec	(subprocess; sandboxed)
code_gen.py	code_gen	code_edit

Network

http.py	http_get	http_post
web_search.py	web_search	

Music

music_xml.py	music_xml_info	music_xml_from_tinynotation	music_xml_transpose
midi.py	midi_info	midi_from_notes	

Audio rendering

render.py	render_midi	(fluidsynth · sfizz · liquidsfz · ffmpeg)			
presets.py	preset_list	preset_get	preset_upsert	preset_delete	

Perception

audio_analyze.py	audio_analyze	
vision.py	vision	
tts.py	tts	(feature-flagged: FEATURE_TTS)

Meta

tool_meta.py	tool_create	tool_publish	tool_promote_to_source	
	tool_remove	tool_list_dynamic	tool_inspect	

Policy filter · policy.yaml

```
tools:   per-name mode/scope
         feature_flag env gate
toolsets: bundled groups
         (core/fs/shell/web/code
          music/audio/vision/meta)
default:  allow
auto_approve:  true

budgets:
  per_turn.tool_calls: 32
  per_turn.wall_clock_sec: 300
  per_session.tokens: 2_000_000
  per_session.usd:    10.0

tool_calls  doubles as max_rounds
in the controller loop.
```

Sandbox / ToolContext · actions/

```
sandbox.py:
  resolve_in_workspace(p, sid)
    → strict session-only
  resolve_in_safe(p, sid)
    → + data/ + extras

JG_OS_SANDBOX=1  (macOS):
  wraps shell / python_exec /
  dynamic-tool subprocesses
  with sandbox-exec
  + data/sandbox/jg.sb

context.py:
  contextvar current() →
  session id + turn idx
```

Dynamic tools (3-tier authoring)

The agent can author its own tools mid-session. Three promotion tiers — session-local → globally published → first-class source.

Tier 1 · session-only

```
trigger:   tool_create(name, description,
                       input_schema, source)

1. validate name (snake_case)
2. validate JSON schema
3. write source:
   workspace/sessions/<sid>/tools/<name>.py
4. construct DynamicSpec:
      name · description · schema
      source · sha256 · execution_mode
5. attach to session's
   DynamicRegistry
6. recompute  controller.allowed

EXECUTION:
  default mode = subprocess
    `python -I <tool.py>`
    cwd = workspace/sessions/<sid>/
    stdin  = JSON kwargs
    stdout = JSON result
    timeout = python_exec policy
    sandboxed if JG_OS_SANDBOX=1
  mode = inproc  (trusted helpers)

lifetime:
  vanishes when session ends
  (unless promoted)
```

tool_publish(name)

Tier 2 · globally published

```
trigger:   tool_publish(name)

1. resolve DynamicSpec from session
2. UPSERT INTO generated_tools
   ( name        unique         )
   ( description input_schema   )
   ( source      sha256         )
   ( scope='global'             )
   ( owner_session_id           )
   ( version, deprecated, meta  )

hydration (every step()):
  store.load_all_specs() →
  DynamicRegistry pulls all
  global rows from generated_tools

→ every future session sees it
  without restart

removal:
  tool_remove(name)
    soft-delete  (deprecated=True)

inspection:
  tool_list_dynamic / tool_inspect
```

tool_promote_to_source(name)

Tier 3 · first-class source

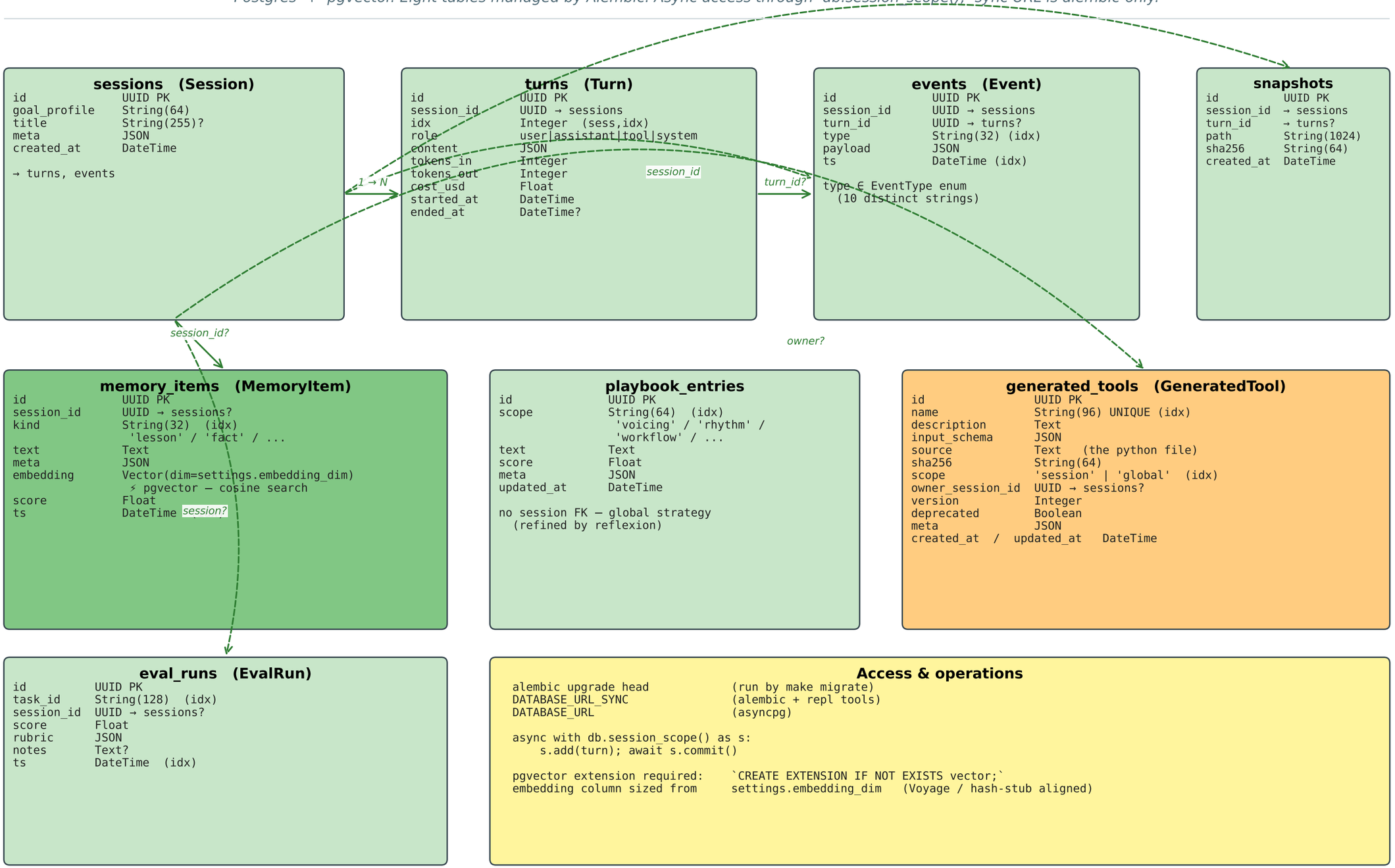
```
trigger:   tool_promote_to_source(name)

1. read source from generated_tools
2. write to:
   src/jazz_guru/actions/tools/<name>.py
3. ensure register_all() will import it
   (auto-detected by directory glob)

STATUS:  needs server / worker restart
   to take effect.

  reasoning:
  Tier 3 changes the codebase.
  Tier 2 stays runtime-reversible.
  Both are surfaced via the same
  tool_use interface to the LLM, but
  the operator decides when to bake
  a generated tool into the source.

GeneratedTool row remains as the
  historical record  (version bump
  possible).
```

Invocation path (every tier)

```
LLM emits  tool_use   →   ActionController dispatch   →   ToolRegistry / DynamicRegistry merge
   →   Pydantic validate(input)   →   subprocess  (`python -I`)  /  inproc call   →   JSON tool_result
      ↓                                                                  ↑
      sandbox-exec wrap (optional)                                  on_event(tool_result)
```

Database schema

Postgres + pgvector. Eight tables managed by Alembic. Async access through db.session_scope(); sync URL is alembic-only.



Memory subsystem

How the agent recalls. pgvector cosine search + Voyage embeddings (or hash-stub) + rolling history summarization.

MemoryStore · memory/store.py

```
async def search(
    query: str,
    k: int = 8,
    session_id: UUID | None = None,
    kind: str | None = None,
  ) -> list[MemoryItem]:
    emb = await embeddings.embed(query)
    SELECT *,
      1 - (embedding <=> :emb) AS sim
    FROM memory_items
    [WHERE kind=:kind AND ...]
    ORDER BY sim DESC  LIMIT :k

async def write(text, kind='lesson',
                score=0.5, meta={}):
    emb = await embeddings.embed(text)
    INSERT INTO memory_items (...)
    emit  memory_write

failures are caught + logged
  → an embedding outage doesn't
    break a turn.
```

embed(text)

embeddings.py

```
Voyage AI client
  model: per settings.embedding_model
  dim:   settings.embedding_dim

fallback when VOYAGE_API_KEY is blank:
  hash-stub  (deterministic)
    sha256 → np.float32 vector
    same dim

tests rely on hash-stub
  → no network in CI

single async  embed(text)  surface
```

state/externalize.StateDoc

```
@dataclass
class StateDoc:
    summary:       str
    open_threads:  list[str]
    artifacts:     list[str]
    last_critique: str | None

load_latest(sid):
  read  workspace/state/<sid>/latest.json
  + walk workspace/sessions/<sid>/
  to enumerate artifacts.

render_markdown(): the block injected
  into the system prompt by
  ContextBuilder.
```

latest.json

summarizer.py

```
rolling history compression:

input:   chat history above a
         configurable cutoff
output:  short LLM-written summary
         held as a system note

keeps  token budget bounded for
  long-running sessions without
  losing the gist.

called from ContextBuilder when
  history exceeds threshold.
```

Snapshots

```
written every turn end + at
  reflexion completion:

workspace/state/<sid>/
  latest.json                ← current
  turn_NNNNN.json            ← history

payload:
  summary  open_threads
  artifacts  last_critique
  last_plan  last_score

Snapshot row in DB references
  path + sha256.
```

Read-path on every turn

```
ContextBuilder.build(...)
    │
    ├─ MemoryStore.search(query)        →   semantic hits  (pgvector cosine)
    ├─ select PlaybookEntry  ORDER BY score LIMIT N
    ├─ state.load_latest(sid)           →   StateDoc.render_markdown()
    └─ summarizer.compact(history)      →   bounded history block
                  ↓
        system_block  +  messages[]   →   llm.complete
```

Distillation / Reflexion

Offline learning loop. Runs after a session — summarises, critiques, and distils memory + playbook + snapshot updates.

Trigger A · async (default)

```
POST /sessions/{id}/distill
      (or  jazz-guru distill <sid>)

→ enqueue   reflexion_job
      on Redis  /  RQ queue
  consumed by  make worker  (worker.py)
  caller returns immediately.
```

Trigger B · sync inline

```
jazz-guru distill <sid> --sync
POST /distill?sync=true

→ run_reflexion(...)  is invoked
  inline in the request handler;
  no Redis hop.
```

distillation/reflexion.run_reflexion(session_id)

```
1.  summarize_transcript(session_id)
      collapse Turn rows + key Events into a compact transcript

2.  build reflexion prompt:
      INPUT:
        · session goal block
        · transcript summary
        · current StateDoc
        · artifact list
      INSTRUCTIONS:  emit STRICT JSON only.

3.  llm.complete(...)  →  parse JSON

4.  apply writeback  (see right →)

5.  emit  reflexion  event
```

strict JSON

JSON contract · writeback targets

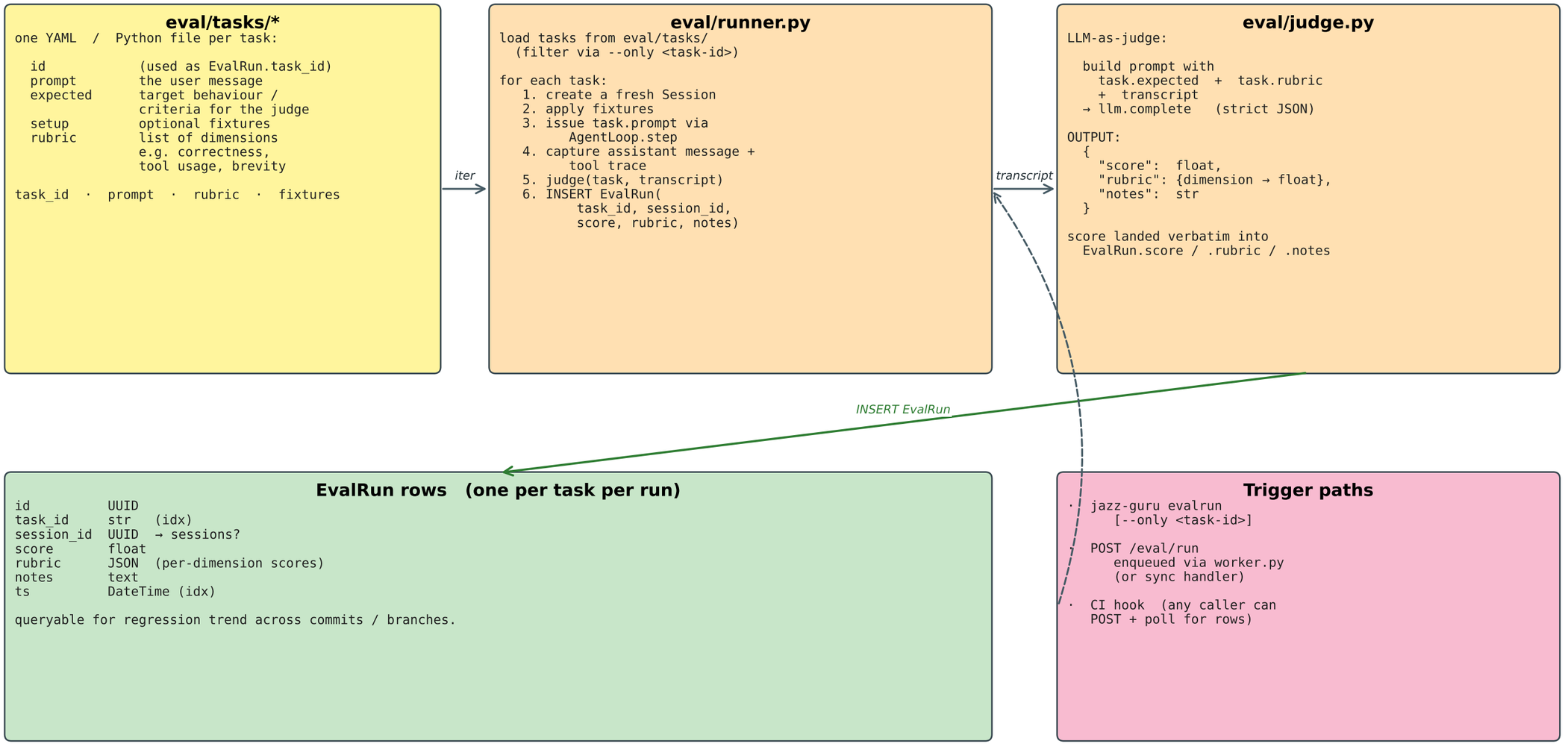
```
{
  "score":   0.0 - 1.0  (clamped)
  "critique":  str
  "revised_plan":  str
  "open_threads":  [str]
  "memory_writes": [
    { "text": str, "kind": str }
  ]
  "playbook_entries": [
    { "scope": str,
      "text":  str,
      "score": float }
  ]
}

WRITEBACK:
  memory_writes      →  MemoryItem
    (score=0.7,  kind from entry)

  playbook_entries   →  PlaybookEntry
    (upsert by scope+text)

  snapshot           →  state/<sid>/
                          latest.json
    fields: summary,
      open_threads, artifacts,
      last_critique,
      last_plan, last_score
```

Feedback loop (how reflexion lands in the next turn)

```
next  AgentLoop.step()  starts  →  ContextBuilder.build:
      ├─ memory.search(query)          →   sees the new MemoryItem  (semantic recall)
      ├─ select PlaybookEntry          →   sees the upserted strategy entry
      └─ state.load_latest()           →   StateDoc carries  summary / open_threads / last_critique
                                           into the system prompt

Both auto-trigger ideas:  POST /distill on session end,  or  a Redis schedule from the worker.
```

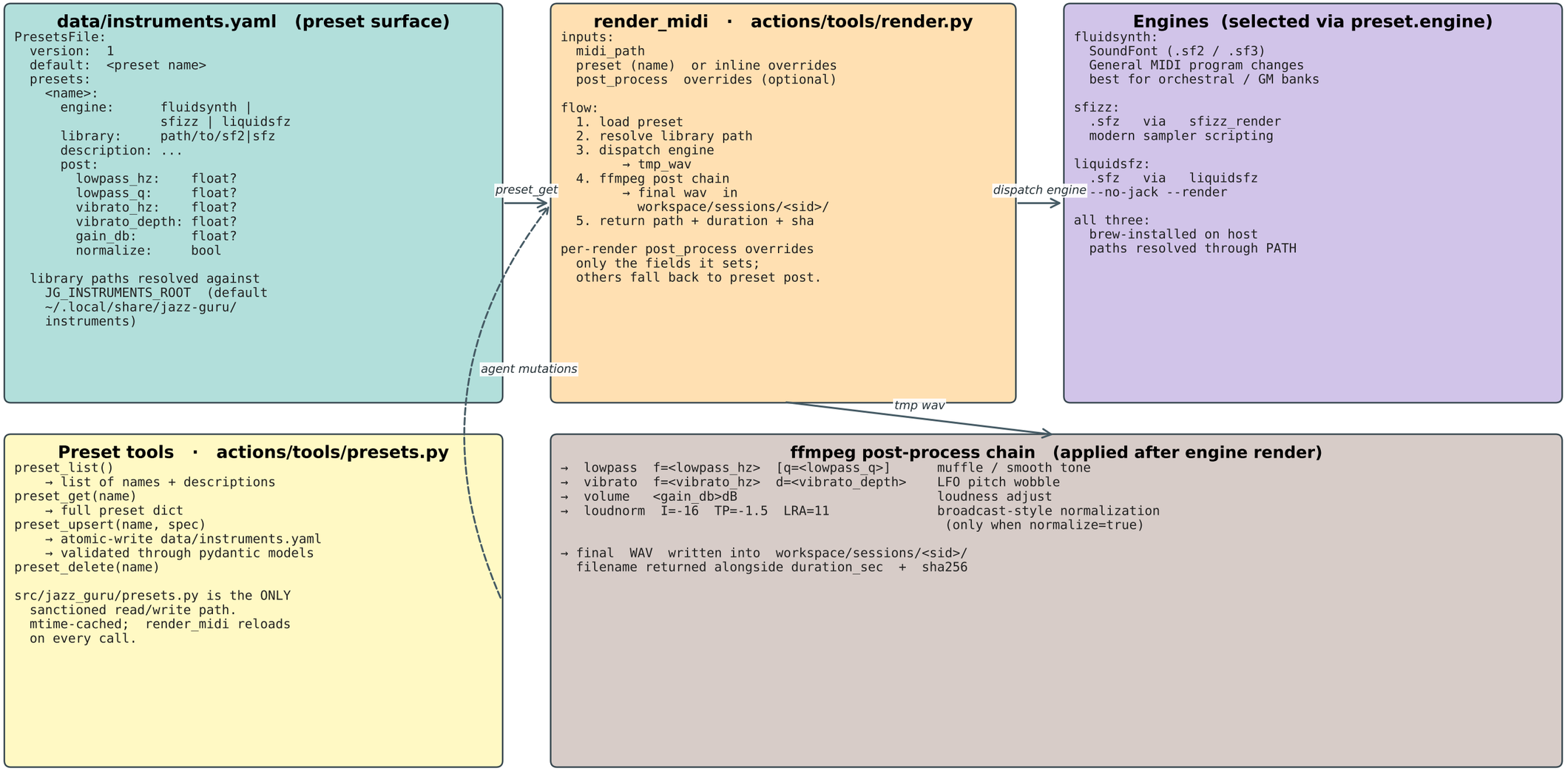
Eval / regression suite

Task replay + LLM-as-judge. Persists scored rubrics into the EvalRun table.



Audio rendering pipeline

render_midi → engine select → SF2/SFZ synth → ffmpeg post chain → audio file.



Configuration sources

Three loaders, all funnelled through src/jazz_guru/config.py with an lru_cache. Restart needed to pick up changes in long-running processes.

.env · pydantic-settings

```
loaded by  pydantic_settings.BaseSettings
  src/jazz_guru/config.py

Settings(
  ANTHROPIC_API_KEY:  str   (required)
  VOYAGE_API_KEY:     str = ''
    blank → hash-stub embeddings
  JG_API_KEY:         str = ''
    set → /chat etc. require
          X-API-Key header
  JG_OS_SANDBOX:      bool = False
  JG_INSTRUMENTS_ROOT: Path
  JG_SAFE_EXTRA_PATHS: list[Path]
  database_url(_sync): str
  redis_url:          str
  embedding_model:    str
  embedding_dim:      int
  workspace_dir / state_dir /
  trace_dir / data_dir
)

get_settings()  → cached singleton
```

config/goal.{md,yaml}

```
goal.md  —  freeform prose preamble
  written by the operator;
  rendered verbatim into system
  prompt.

goal.yaml  —  structured fields:
  profile:           str
  objectives:
    - id:      str
      text:    str
      weight:  float
  constraints:       [str]
  success_criteria:  [str]
  style:
    voice:               str
    jargon:              str
    notation_priority:   str
    ...

GoalConfig.render_system_block()
  concatenates both and prepends
  to every system prompt.
```

config/policy.yaml

```
version:  1
default:        "allow"
auto_approve:   true
default_max_result_bytes:  10000

tools:
  <name>:
    mode:               allow|deny
    scope:              str
    timeout_sec:        int
    mem_mb:             int
    max_result_bytes:   int
    feature_flag:       FEATURE_X

toolsets:
  core / fs / shell / web / code /
  music / audio / vision / meta

budgets:
  per_turn:
    tool_calls:       32
    wall_clock_sec:  300
  per_session:
    tokens:    2_000_000
    usd:         10.0
```

How they fuse

```
Every  llm.complete  call assembles its system prompt as:

    GoalConfig.render_system_block()        ← goal.md + goal.yaml
  + _TOOL_CREATION_HINT                     ← static, in context/builder.py
  + StateDoc.render_markdown()              ← per-session snapshot
  + playbook entries                        ← top-N from playbook_entries
  + retrieved memory                        ← MemoryStore.search()
  + (history summary)                       ← summarizer.compact()

Policy filtering is applied per turn:
    ActionController.allowed =
       [ spec for spec in registry.specs()
         if policy.for_tool(spec.name).mode == 'allow'
         and (not p.feature_flag or os.getenv(p.feature_flag)) ]

CACHE  notes:
    get_settings  is  lru_cache.   If you edit  .env  or any YAML in a long-running
    server / worker, restart the process  (or call get_settings.cache_clear()).
```

Observability

Every controller step emits an event. Three subscribers consume the stream — trace JSONL, WebSocket clients, and the Postgres event log.

Event types · state.schema.EventType

```
turn_start          AgentLoop.step start
turn_end            step complete
llm_request         payload sent to Anthropic
llm_response        full reply (with stop_reason)
tool_use            controller dispatching a tool
tool_result         tool returned a JSON result
memory_write        MemoryItem inserted
memory_read         memory.search executed
reflexion           run_reflexion completed
error               anything wrapped via try/except
                    (budget exhaustion, tool fail,
                     LLM rate-limit retries, ...)
```

enum values

ActionController.on_event

```
def on_event(
    event_type: str,
    payload: dict,
    *,
    session_id: UUID,
    turn_id: UUID | None,
) -> None:
    for subscriber in subscribers:
        subscriber(event_type,
                   payload, ...)

subscribers attached by AgentLoop /
  server during step() construction.
```

JSONL

logging/trace.TraceWriter

```
appends one JSON object per event:

  workspace/traces/<sid>.jsonl

format:
  {"ts": iso, "type": ...,
   "turn_id": ..., "payload": ...}

consumed by:
  jazz-guru viewer       :8765
  jazz-guru trace <sid>
    (dumps file to stdout)
```

broadcast

INSERT

reads

WebSocket · WS /ws/sessions/{id}/chat

```
auth via subprotocol header:
  Sec-WebSocket-Protocol:
    bearer, <JG_API_KEY>
  server echoes 'bearer'
  (Starlette middleware doesn't run
   for WS — auth.require_ws() is
   called explicitly.)

server pushes:
  every controller event,
  in original order

consumers:
  Textual TUI live view
  custom SDK clients
```

state/event_log.py

```
INSERT INTO events (...)
for every emitted event:

  type, payload (JSON),
  session_id, turn_id?, ts

indexed by type + ts.

queryable for replay,
  postmortems, telemetry.
```

logging/viewer/ · trace viewer

```
launched via
  jazz-guru viewer  → :8765

tiny uvicorn app:
  lists workspace/traces/*.jsonl
  tails the chosen file in browser
  pretty-prints each event by type

useful when you're not running
  the full FastAPI server but still
  want a live view of an agent run.
```

Plus structlog (app-level)

```
structlog is configured at module import (logging/__init__.py)  →  JSON / console renderer.
Used for everything outside the event stream: setup errors, retries inside llm.complete,
config loading, alembic migrations, FastAPI access logs (via uvicorn).
```

Filesystem & macOS sandbox

The agent's writable footprint is confined to workspace/. On macOS JG_OS_SANDBOX=1 wraps subprocesses with sandbox-exec.

Directory tree (relative to project root)

```
jazz-guru/
  .env                        secrets · feature flags
  alembic/                    DB migrations
  alembic.ini
  Makefile                    setup · server · worker · tui · viewer
  pyproject.toml              ruff / mypy / pytest config

  config/
    goal.md                   prose preamble
    goal.yaml                 structured objectives / constraints
    policy.yaml               per-tool gates + budgets

  data/
    instruments.yaml          preset surface (agent-mutable)
    sandbox/jg.sb             sandbox-exec profile (macOS)

  src/jazz_guru/
    cli.py · server.py · auth.py · llm.py · presets.py · worker.py
    config.py · db.py
    harness/   actions/   context/   memory/   state/
    distillation/   eval/   client/   logging/   web/

  tests/unit/                 pytest (asyncio_mode = 'auto')

  workspace/                  ← the only writable runtime root
    sessions/<sid>/           artifacts · uploads
    sessions/<sid>/tools/     tier-1 dynamic tool files
    state/<sid>/latest.json   current snapshot
    state/<sid>/turn_*.json   per-turn snapshots
    traces/<sid>.jsonl        event trace

~/.local/share/jazz-guru/     (JG_INSTRUMENTS_ROOT)
  instruments/                SF2 / SFZ libraries
```

data/sandbox/jg.sb · macOS sandbox-exec profile

```
wrapped automatically when
  JG_OS_SANDBOX=1   for:
      shell · python_exec · dynamic-tool subprocesses

invocation:
  sandbox-exec -f data/sandbox/jg.sb \
    -D WORKSPACE=/abs/workspace/sessions/<sid> \
    -D DATA=/abs/data \
    <argv...>

rules summary:
  deny  everything by default
  allow process startup:
      fork · exec · signal · mach-lookup
      sysctl-read · ipc-posix-shm
  allow file-read:
      /usr/lib  /usr/share  /System/Library
      Frameworks  /opt/homebrew  /usr/local
      /private/var/db/dyld
      pyenv  +  .venv install roots
      {WORKSPACE}  {DATA}
  allow file-write:
      {WORKSPACE}
      /private/tmp · /private/var/folders
  allow network*
```

Sandbox helpers · actions/sandbox.py

```
resolve_in_workspace(path, sid)
  → strict: only paths inside
    workspace/sessions/<sid>/
  → used for WRITES and any user-
    supplied path that should stay
    inside the session.

resolve_in_safe(path, sid)
  → read-oriented: above set + data/
    + any JG_SAFE_EXTRA_PATHS.
  → used by tools that legitimately
    read project data (e.g. presets).

tools that touch project data outside
    the session  →  expose a typed
    surface (e.g. presets tools).
    NEVER require the agent to use
    fs_write / shell / python_exec.
```

Server surface & clients

FastAPI HTTP/WS endpoints, optional X-API-Key middleware, plus the CLI and TUI.

FastAPI routes · src/jazz_guru/server.py

```
GET   /                          link page
GET   /favicon.ico
GET   /health                    liveness probe
GET   /goal                      rendered goal + objectives

POST  /sessions                  create  → {id}
POST  /sessions/{id}/chat        send a message
POST  /sessions/{id}/distill     reflexion  (sync ?sync=true | async)

POST  /eval/run                  run regression suite
POST  /memory/search             vector search

POST  /uploads/{id}              stream upload → workspace
GET   /artifacts/{id}            list session artifacts
GET   /artifacts/{id}/{path}     download a single artifact

WS    /ws/sessions/{id}/chat     streaming tool events
                                  subprotocol: bearer + token

GET   /ui/*                      static web UI (web/static/)
```

Auth · auth.py

```
optional X-API-Key middleware:
    enabled iff   JG_API_KEY  is set in env
    response:    401  on missing / mismatching key

always-exempt paths:
    /     /health     /docs     /redoc     /openapi.json
    /favicon.ico      /ui   /ui/*

WebSocket auth:
    Starlette middleware does NOT run for WS.
    Each WS handler calls   auth.require_ws(token)
    using the Sec-WebSocket-Protocol header.
```

CLI · src/jazz_guru/cli.py

```
jazz-guru                (entrypoint)
  info                   show resolved config
  ping                   verify ANTHROPIC_API_KEY
  goal                   print rendered goal block
  tools                  list registered tools + schemas
  new-session            create + print a Session id
  chat "<msg>" [--session <uuid>]
  distill <sid> [--sync]  reflexion loop
  evalrun [--only <id>]   regression suite
  trace <sid>            dump JSONL trace
  viewer                 trace viewer on :8765
  tui                    Textual TUI (server required)
  mic-devices            list PortAudio inputs

jazz-guru-server         uvicorn shim
jazz-guru-worker         RQ worker shim
```

Client-side ecosystem

```
Textual TUI                src/jazz_guru/client/tui.py
    PortAudio mic capture  +  live event tail   ·   subscribes to WS /ws/sessions/{id}/chat

Typed SDK                  src/jazz_guru/client/sdk.py
    plain HTTP wrappers for  /sessions /chat /distill /memory/search /eval/run /artifacts
    plus the WS streaming helper

Static web UI              src/jazz_guru/web/static/        served at /ui/
    minimal HTML+JS chat surface for casual use

Trace viewer               src/jazz_guru/logging/viewer/    served on :8765
    independent of the main server; useful when running offline batch sessions

External CI / scripts      any HTTP client
    POST  /eval/run  to gate releases; poll  EvalRun  rows from Postgres
```